

Spark Homes Repair Estimator

Developer Contest Submission — Priyank Mandal

1. Most Interesting Design / UX Decision

The most interesting decision was avoiding full-DOM rebuilds for rapid interactions. In a single-file vanilla JS app, the natural pattern is innerHTML teardown on every state change. Early in development, checking a checkbox caused the entire 1000+ line item list to rebuild, losing focus, scroll position, and producing visible flash on slow devices. I introduced a surgical DOM update layer: `toggleItem`, `setQty`, and `setPrice` now update only the affected row class, line total, group badge, section cost breakdown, progress bar, and grand total — without touching any other DOM nodes. This eliminated all jank while keeping the codebase a single HTML file. The approach uses predictable element IDs derived from the item's scoped key (`roomId::itemId`) so the JavaScript can find and patch individual cells regardless of how many rooms or items exist.

2. What Is Broken or Fragile

Three areas are worth flagging. First, custom items are indexed by array position in `proj.customItems`. Deleting a custom item splices the array, shifting all subsequent indices. The event delegation handles this by using the real-time array index via data-action attributes, but any stale references (e.g., in-progress user interactions) could target the wrong item. A production app would use UUID-keyed maps. Second, the cost breakdown section relies on `renderItems` generating DOM IDs that `updateUI` patches. If the breakdown isn't in the DOM (e.g., during the initial empty state), updates silently fail — the totals won't appear until the next full render. Third, photo capture uses the standard `<input type="file" capture="environment">` pattern. On iOS Safari this sometimes presents a file picker instead of launching the camera directly, which is a known platform limitation without a clean workaround.

3. Creative Addition — Deal Analyzer

The creative addition is a Deal Analyzer that answers the core business question: "Does this deal make money?" It prompts the user for ARV (After Repair Value), purchase price, and four real-world cost parameters (closing costs percentage, monthly holding cost, months held, selling costs percentage). It automatically pulls the running repair estimate and computes: total cost basis, estimated profit, ROI percentage, and margin percentage. Results are color-coded (green for profit, red for loss). I chose this feature because the brief emphasizes that "the most important number is the repair estimate" but in practice that number is meaningless without full deal context — an acquisition agent standing in a house needs to know not just "repairs cost \$45K" but "with a \$200K ARV and \$120K purchase, I'm at 14% margin, which is above our hurdle rate." All inputs are persisted per-project so they survive page reloads and device swaps. A secondary creative addition is the CSV price list import: agents can upload the official `Pricing List.csv` to bulk-update all 108 standard prices at once, addressing the brief's global pricing requirement.

4. What I Would Ship Next (Two More Days)

With two additional days, I would focus on three things. First, add photo-to-item association: allow agents to tag a photo to a specific room or line item so the export shows which photo documents which repair. This would require a lightweight annotation overlay on the photo grid. Second, add an offline-first sync layer: since the PWA works offline but uses `localStorage/IndexedDB`, a simple JSON export/import mechanism would let agents transfer estimates between devices without a backend — useful when a colleague takes over a property walkthrough mid-inspection. Third, add a PDF report export alongside the Excel/ZIP export, since contractors on site often prefer a printable summary they can mark up by hand. This would reuse the existing item data and generate a clean single-page scope-of-work PDF using the same approach as the deal analyzer.

5. Role of AI Tools

AI tools (Claude Code) were used throughout development for: generating the initial 108-item data model from the CSV, writing the `IndexedDB` photo storage wrapper, implementing the minimal ZIP and XLSX generators for offline export, debugging service worker edge cases, creating the Playwright test suite, and generating this writeup. The overall architecture, event delegation system, room/group data model, surgical DOM update layer, and deal analyzer logic were designed and written by me, with AI assisting in implementation. AI was most valuable for rapidly iterating on the service worker caching strategy and for catching subtle edge cases in the event delegation router that would have been easy to

miss.